

|                                           |                                                                                                                                                                                                                                                                                                    |
|-------------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Project Title</b>                      | <b>Sales Intelligence Hub</b>                                                                                                                                                                                                                                                                      |
| <b>Skills take away From This Project</b> | <ul style="list-style-type: none"> <li>• Python</li> <li>• MySQL Database Design</li> <li>• SQL Queries &amp; Aggregations</li> <li>• Triggers &amp; Generated Columns</li> <li>• Streamlit Web Application</li> <li>• Payment Tracking Logic</li> <li>• Relational Database Management</li> </ul> |
| <b>Domain</b>                             | <b>Sales Analytics &amp; Financial Tracking System</b>                                                                                                                                                                                                                                             |

### Problem Statement:

Organizations operating across multiple branches often face challenges in tracking sales, managing payment collections, monitoring pending amounts, and maintaining structured financial records. Manual tracking methods may result in data inconsistencies, duplicate entries, incorrect payment calculations, and lack of transparency in revenue sharing.

The objective of this project is to design and implement a structured **Branch-Based Sales Management System** using MySQL, Python, and Streamlit.

### The system will:

- Store branch-wise sales records
- Automatically calculate net sales
- Support split payment tracking
- Automatically update received and pending amounts using database triggers
- Provide an interactive Streamlit dashboard for Admin users

This solution ensures accurate financial reporting, automated calculations, and real-time sales monitoring.

## Business Use Cases:

- Track branch-wise sales performance
- Monitor received vs pending payments
- Automatically calculate net revenue after deductions
- Manage split payments for customers
- Identify high-performing branches
- Analyze sales trends by date
- Monitor revenue-sharing payouts
- Generate financial summaries for business decision-making

## DATABASE TABLE STRUCTURE

### 1. branches Table

#### Purpose:

Stores information about different business branches.

#### Table Structure:

| Column Name       | Data Type    | Description                               |
|-------------------|--------------|-------------------------------------------|
| branch_id         | INT (PK, AI) | Unique identifier for each branch         |
| branch_name       | VARCHAR(100) | Name of the branch (e.g., Chennai, Delhi) |
| branch_admin_name | VARCHAR(100) | Name of the branch administrator          |

#### Key Points:

- branch\_id is Primary Key
- Referenced by customer\_sales and users table
- Maintains branch-level organization

### 2. customer\_sales Table (Main Financial Table)

#### Purpose:

Stores all customer sales transactions.

### Table Structure:

| Column Name     | Data Type                           | Description                      |
|-----------------|-------------------------------------|----------------------------------|
| sale_id         | INT (PK, AI)                        | Unique sale transaction ID       |
| branch_id       | INT (FK)                            | References branches table        |
| date            | DATE                                | Date of the sale                 |
| name            | VARCHAR(100)                        | Customer name                    |
| mobile_number   | VARCHAR(15)                         | Customer mobile number           |
| product_name    | VARCHAR(30)                         | (DS/DA/BA/FSD)                   |
| gross_sales     | DECIMAL(12,2)                       | Total sale amount                |
| received_amount | DECIMAL(12,2)                       | Auto-updated via triggers        |
| pending_amount  | DECIMAL(12,2)<br>(Generated Column) | gross_sales -<br>received_amount |
| status          | ENUM('Open','Close')                | Indicates sale completion        |

 **Key Logic:**

- pending\_amount is auto-calculated
- received\_amount updated automatically via triggers
- branch\_id maintains relational integrity

### 3. users Table (Admin Only Role)

#### Purpose:

Manages system login access (**Super Admin** role and **Admin** role supported).

#### Table Structure:

| Column Name | Data Type                    | Description                          |
|-------------|------------------------------|--------------------------------------|
| user_id     | INT (PK, AI)                 | Unique user ID                       |
| username    | VARCHAR(100)                 | Admin username                       |
| password    | VARCHAR(255)                 | password                             |
| branch_id   | INT (FK)                     | Assigned branch                      |
| role        | ENUM('Super Admin', 'Admin') | Super Admin and Admin role supported |
| email       | VARCHAR(255) (UNIQUE)        | Super Admin and Admin email ID       |

#### Key Points:

- The **Super Admin** can create and view data for **all branches**.

The **Admin** can create and view data **only for their respective branch**.

- Email must be unique

#### 4. payment\_splits Table

##### Purpose:

Handles multiple payments for a single sale.

| Column Name    | Data Type     | Description                   |
|----------------|---------------|-------------------------------|
| payment_id     | INT (PK, AI)  | Unique payment transaction ID |
| sale_id        | INT (FK)      | References sales table        |
| payment_date   | DATE          | Date of payment               |
| amount_paid    | DECIMAL(12,2) | Payment amount                |
| payment_method | VARCHAR(50)   | Cash / UPI / Card             |

##### Table Structure:

##### Key Logic:

- Multiple payment records can exist for one sale
- Trigger automatically updates received\_amount in sales table

- Supports partial payments
  - Ensures financial consistency
- 

## AUTOMATION LOGIC (TRIGGERS)

Triggers will run:

- AFTER INSERT on payment\_splits

Each trigger will:

1. Calculate total amount\_paid for that sale
2. Update received\_amount in customer\_sales table
3. Automatically adjust pending\_amount

This ensures:

- No manual financial calculation
  - Accurate pending tracking
  - Real-time payment update
- 

## TABLE RELATIONSHIP SUMMARY

branches (1) → (Many) customer\_sales  
customer\_sales (1) → (Many) payment\_splits  
branches (1) → (Many) users

**Approach:**

### 1. Database Design (MySQL)

Create a structured database: **Sales Management System**

**Design Relational Tables:**

- **branches**
- **customer\_sales**
- **users** (Super Admin and Admin only)
- **payment\_splits**

#### Implement:

- Primary Keys
- Foreign Keys
- Data Constraints
- ENUM status field ('Open', 'Close')
- UNIQUE constraint on mobile number
- Referential integrity between tables

#### Use Generated Column:

- $\text{pending\_amount} = \text{gross\_sales} - \text{received\_amount}$

This ensures automatic calculation of outstanding balance.

This system supports:

- ✓ Multi-branch data storage
- ✓ Centralized monitoring
- ✓ Full access for **Super Admin** across all branches

## 2. Trigger Implementation

Implement triggers on **payment\_splits** table:

- AFTER INSERT

#### Trigger Responsibilities:

- Recalculate total received amount

- Update received\_amount in sales table
- Automatically adjust pending\_amount
- Maintain financial accuracy

This prevents manual calculation errors and ensures data consistency.

---

### 3. Streamlit Application Development

Develop a **centralized dashboard using Streamlit** that supports **role-based access control** for Super Admin and Admin users.

---



#### Login Module (Mandatory)

- Implement secure username and password authentication for system access.
  - Validate login credentials using the users table in the database.
  - Allow access to the dashboard only after successful authentication.
  - Implement session-based login management to maintain user authentication during the session.
  - Provide a logout option for users to securely exit the system.
  - Prevent unauthorized users from accessing the dashboard or application features without proper login.
- 



#### Role-Based Access Control

The system supports **two user roles**:

##### Super Admin

- Can access data from all branches.
- Can view all sales records across the system.
- Can view all payment split details.



- Can access complete business analytics and reports.
- Can monitor overall business performance across branches.

## Admin

- Can create sales entries for their assigned branch.
- Can add payment split records for sales belonging to their branch.
- Can view sales data only for their branch.
- Can track pending payments for their branch.
- Can access branch-specific reports and analytics.

## Key Points

- The Super Admin can create and view data for all branches.
- The Admin can create and view data only for their respective branch.
- Role-based access control ensures secure and organized data management.
- The login system prevents unauthorized access to the application.

## Dashboard Features

- Add a new sales entry.
- Super Admin can add sales for any branch.
- Admin can add sales only for their assigned branch.
- Add payment split for recorded sales.
- View sales report.
- Super Admin can view sales data from all branches.
- Admin can view sales data only for their branch.
- Branch filter option for analyzing sales by branch.
- View pending payments.
- View sales status (Open vs Close).
- Payment method summary (Cash / UPI / Card).
- Execute predefined SQL queries (Mandatory).
- Display Financial KPI summary such as total sales, received amount, and pending amount.

## 4. Insights & Reporting

The system should provide:

- Overall business revenue.

- Branch-wise sales comparison report.
- Total received amount vs total pending amount.
- Pending collection percentage.
- Payment method analysis (Cash / UPI / Card).
- Sales trends and growth tracking.
- Branch performance summary dashboard.

### This project ensures:

- Multi-branch sales monitoring system
- Accurate payment tracking using payment splits
- Automated financial calculations using triggers and generated columns
- Secure authentication with role-based access control
- Structured database management using MySQL
- Interactive reporting dashboard using Streamlit



### SQL QUESTIONS (Total: 20)

**Note:** Before performing data analysis using SQL queries, the demo data must be inserted into the MySQL database.

### Datasets: Sales Management System Datasets

#### ♦ Basic Queries - Answer any 4 questions.

1. Retrieve all records from the **customer\_sales** table.
2. Retrieve all records from the **branches** table.
3. Retrieve all records from the **payment\_splits** table.
4. Display all sales with status = 'Open'.
5. Retrieve all sales belonging to the **Chennai branch**.

---

#### ♦ Aggregation Queries - Answer any 4 questions.

6. Calculate the total **gross sales** across all branches.
7. Calculate the total **received amount** across all sales.
8. Calculate the total **pending amount** across all sales.

9. Count the total number of sales per branch.
10. Find the average gross sales amount.

---

♦ **Join-Based Queries - Answer any 4 questions.**

11. Retrieve sales details along with the **branch name**.
12. Retrieve sales details along with total payment received (using payment\_splits).
13. Show branch-wise total gross sales (using JOIN & GROUP BY).
14. Display sales along with payment method used.
15. Retrieve sales along with branch admin name.

---

♦ **Financial Tracking Queries - Answer any 3 questions.**

16. Find sales where the pending amount is greater than 5000.
17. Retrieve top 3 highest gross sales.
18. Find the branch with highest total gross sales.
19. Retrieve monthly sales summary (group by month & year).
20. Calculate payment method-wise total collection (Cash / UPI / Card).

**Results:**

- Structured relational database
- Automatic financial calculations
- Accurate payment tracking system
- Functional Streamlit admin dashboard
- Real-time branch-level reporting
- Clean and normalized database design

### Project Evaluation metrics:

- Proper database normalization
- Correct implementation of triggers
- Accurate financial calculations
- SQL query optimization
- Use of joins and aggregations
- Streamlit functionality & UI clarity
- Clean Python code structure
- Business insight interpretation

### Technical Tags:

Python, MySQL, SQL Triggers, Generated Columns, Financial Analytics, Streamlit Dashboard, Sales Management System

### Project Deliverables:

- Complete MySQL database schema
- Trigger implementation
- Sample data insertion
- Any 15 SQL analytical queries
- Python-MySQL connection script
- Streamlit dashboard
- Documentation file

### Project Guidelines:

- Follow proper database normalization
- Implement triggers correctly
- Ensure no manual update of received\_amount
- Maintain clean Python code
- Build structured Streamlit interface
- Provide clear documentation

### Timeline:

The project must be completed and submitted **within 14 days from the assigned date.**

## References:

|                                                                                    |                                                                                                                            |
|------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| Project Live Evaluation Guidelines                                                 |  Project Live Evaluation                  |
| Project Excellence Series: Guided Learning & Problem Solving [Python & SQL](Tamil) |  Project Excellence Series: Guided L...   |
| Capstone Explanation Guideline                                                     |  Capstone Explanation Guideline           |
| GitHub Reference                                                                   |  How to Use GitHub.pptx                   |
| Streamlit Reference English                                                        | <a href="#">Streamlit Recordings</a>                                                                                       |
| Streamlit Reference Tamil                                                          | <a href="#">Streamlit Recording Tamil</a>                                                                                  |
| Project Orientation (English)                                                      |                                                                                                                            |
| Project Orientation (Tamil)                                                        |  Project Orientation Session :Sales ... |

## PROJECT DOUBT CLARIFICATION SESSION ( PROJECT AND CLASS DOUBTS)

**About Session:** The Project Doubt Clarification Session is a helpful resource for resolving questions and concerns about projects and class topics. It provides support in understanding project requirements, addressing code issues, and clarifying class concepts. The session aims to enhance comprehension and provide guidance to overcome challenges effectively.

**Note:** You can join the doubt session using the link provided in document link given below

**Timing:** Monday-Saturday (3:30PM to 4:30PM)

**Session link :** [Project Doubt Session \[DS/AIML\]](#)

## LIVE EVALUATION SESSION (CAPSTONE AND FINAL PROJECT)

**About Session:** The Live Evaluation Session for Capstone and Final Projects allows participants to showcase their projects and receive real-time feedback for improvement. It assesses project quality and provides an opportunity for discussion and evaluation.

**Note:** This form will Open only on Saturday (after 2 PM ) and Sunday on Every Week

**Timing:** Monday-Saturday (11:30 AM to 12:30 PM & 05:30PM)

**BookingLink :** <https://forms.gle/1m2Gsro41fLtZurRA>

|                           |                     |                     |
|---------------------------|---------------------|---------------------|
| <b>Created By:</b>        | <b>Verified By:</b> | <b>Approved By:</b> |
| <a href="#">VIGNESH P</a> | Selvamani A         | Nehlath Harmain     |